

OCEAN: Open-Source CXL Emulation for Hyperscale Architecture and Networking

Abstract—Compute Express Link (CXL) 3.0 introduces fabric-level memory pooling, yet the absence of production CXL 3.0 hardware and the complexity of multi-host configurations pose significant barriers to the research community. This paper presents OCEAN, a high-fidelity, open-source emulation framework that delivers full CXL 3.0 functionality—including multi-host memory sharing, dynamic capacity management, and coherent pooling across hosts—using QEMU virtualization and custom Linux kernel modules. OCEAN makes two central technical contributions: active latency injection and a software cache coherence layer implementing MESI-state management with distributed snoop filters to maintain correctness across up to 16 hosts. We validate OCEAN against a real CXL Type-3 device. In addition, Case studies on Tigon database workloads quantify the throughput impact of hardware cache coherence — an $11\times$ improvement at full coverage — while a design-space sweep of twelve memory-placement policies on GROMACS demonstrates up to 7% performance variation from policy choice alone.

I. INTRODUCTION

Compute Express Link (CXL) is an emerging interconnect technology that enables systems to attach additional memory through high-speed links (e.g., PCIe), beyond the DRAM installed directly on the motherboard to support growing demand in memory capacities and bandwidths of datacenter-scale workloads. The latest revision, CXL 3.0 [5], introduces fabric-level connectivity that enables coherent memory pooling across multiple hosts using standard load/store semantics. By replacing message-passing and RDMA-based data movement with direct shared-memory access, CXL 3.0 has the potential to fundamentally restructure how datacenter applications manage and share data.

Although CXL 3.0 in principle allows hundreds of hosts to share a unified memory pool, in practice, however, production CXL 3.0 hardware remains unavailable, and existing physical deployments are confined to single-cabinet or rack-scale configurations. Consequently, published evaluations are limited either to proxy microkernels running on a single server [6] or to realistic distributed workloads executed at scales too small to expose cross-host benefits [7]–[10]. At these small scales, CXL’s latency penalty often outweighs its capacity advantage for compute-bound workloads.

Despite CXL’s potential, there are several key challenges that prevent the community from realizing—or even quantifying—the benefits of CXL at scale. (i) Application software remains anchored to the private-memory, message-passing paradigm; collective operations and data layouts have not been restructured to exploit a shared memory pool. (ii) CXL hardware is scarce and expensive: no leading supercomputer or hyperscale datacenter has deployed more than

a handful of CXL-enabled nodes, precluding large-scale experimentation. (iii) Existing emulation and simulation tools [2], [3], [11], [12] are either cycle-accurate but prohibitively slow, or limited to single-host configurations without CXL 3.0 fabric, coherency, or multi-host support. (iv) CXL devices lag behind the specification. No platform exists today that enables researchers to run unmodified applications against a *multi-host* CXL 3.0 fabric at meaningful scale.

In response to these challenges, we present OCEAN, a comprehensive *multi-host* software emulation framework that faithfully emulates CXL 3.0 hosts, memory devices, and fabric switches with full capability support. OCEAN provides a realistic and accessible platform for researchers and developers to explore CXL 3.0 capabilities, develop and test software, and evaluate system designs ahead of hardware availability. OCEAN makes the following contributions:

- 1) **Complete CXL 3.0 emulation with high fidelity:** OCEAN implements the full CXL 3.0 specification, supports fabric management and coherency features, including Global Fabric Attached Memory (GFAM), Multi-Headed Single Logical Devices (MH-SLD) supporting up to 16 hosts, and Dynamic Capacity Devices (DCD) with runtime hot-add/remove. Emulation fidelity is ensured by two mechanisms: *active latency injection*, which stalls each vCPU for the precise duration of the simulated CXL access after compensating for IPC overhead, and a *software cache coherence engine* implementing the MESI protocol with back-invalidation.
- 2) **Scalable multi-host fabric architecture.** OCEAN models a complete CXL 3.0 fabric with multi-level switches supporting port-based routing. A distributed Fabric Manager coordinates topology discovery, address allocation, and dynamic capacity events across all participants. OCEAN implements the multi-host coherency protocols to ensure data consistency across hosts.
- 3) **Transparent software compatibility and portability:** OCEAN is designed to be fully compatible with existing CXL software stacks. It integrates with the Linux kernel’s CXL driver and management tools, allowing *unmodified* applications and system software to run as is on OCEAN as if it ran on real CXL hardware.
- 4) **Evaluation:** In multi-host setting, we validate OCEAN against real CXL Type-3 hardware. Microbenchmarks (LMbench, STREAM, OSU) and full-application case studies – including Tigon database workloads, GROMACS molecular dynamics, and a 12-policy memory-

TABLE I: Comparison between OCEAN and prior CXL emulators/simulators. CXL 3.0 introduces three multi-host features absent from earlier specs: **GFAM MH-SLD**, and **DCD**. Multi-host support enables memory pooling (lower TCO), cross-host coherent sharing (faster collective communication), and elastic capacity rebalancing (higher utilization).

System	CXL Ver.	Full Sys.	Multi-Host	HW Valid.	Type 1/2	Config.	Sim. Err.	Sim. Speed	Category	CXL 3.0 Features	
										GFAM/MH-SLD	DCD
CXLMemSim [1]	—	No	No	No	×	Med	High	Med	SW Sim	×	×
CXL-DMSim [2]	1.1	Yes	No	Yes	×	High	Low	Low	SW Sim	×	×
Mess+gem5	—	No	No	No	×	High	Med	Low	SW Sim	×	×
QEMU [3]	2.0	Yes	No	No	×	High	High	High	SW Emu	×	Partial
emucxl [4]	—	No	No	N/A	×	Low	High	High	HW Emu	×	×
SimCXL	1.1	Yes	No	Yes	×	High	Low	Low	SW Sim	×	×
OCEAN (this work)	3.0	Yes	Yes	Yes	✓	High	Low	High	SW Emu	✓	✓

placement design sweep – demonstrate both emulation fidelity and OCEAN’s utility as a design-space exploration platform for CXL 3.0 software. OCEAN will be released as an open-source platform available to the community.

II. BACKGROUND

A. CXL protocol

CXL has evolved rapidly since 2019 [13]. CXL 1.0/1.1 established cache-coherent processor-accelerator links; CXL 2.0 added memory pooling and switching. CXL 3.0 extends these with four features central to OCEAN’s design. **Global Fabric Attached Memory (GFAM)** allows devices to be shared across multiple hosts through a CXL fabric with unified addressing and coherency. **Multi-Headed Single Logical Device (MH-SLD)** lets a single device expose up to 16 logical interfaces to different hosts—this specification limit directly bounds our multi-host configuration. **Dynamic Capacity Devices (DCD)** support runtime hot-add and hot-remove of memory without rebooting. **Enhanced Fabric Management** provides discovery, configuration, and monitoring of multi-host topologies. CXL 3.0 also doubles link bandwidth to 64 GT/s per lane.

B. Related Work

Table I summarizes how OCEAN compares to existing emulation and simulation approaches. The QEMU project added basic CXL 2.0 device support [3], but covers only single-host scenarios without CXL 3.0 features. NUMA-based platforms [4], [8], [14], [15] emulate CXL memory expansion on real hardware with limited programmability. Cycle-accurate tools such as CXL-DMSim [2] and gem5 [16], [17] offer micro-architectural fidelity but are orders of magnitude too slow for OS-level or application experiments. CXLMemSim [1], ESF [18], and DRackSim [19], [20] provide partial CXL modeling via software interception, yet none support multi-host fabrics. Hardware characterizations [21]–[25] supply the latency and bandwidth targets OCEAN validates against.

Memory disaggregation systems such as LegoOS [26] and Hydra [27], [28] demonstrate the potential of breaking the one-machine-one-memory model via custom RDMA protocols. For HPC workloads specifically, Wahlgren et al. quantify the memory disaggregation requirements using an emulated CXL platform [29], [30]. OCEAN differs by leveraging the CXL standard, meaning applications can in principle migrate

to future CXL hardware without modification. Prior multi-host systems like ScaleMP vSMP and Numascale NumaConnect [31], [32] require proprietary interconnects; CXL 3.0 provides equivalent capability as an open standard.

III. OVERVIEW

Faithfully emulating CXL 3.0 presents three challenges.

Latency, granularity, and compatibility. Software interception of memory accesses stalls the vCPU until the handler responds, introducing microsecond-scale overhead that dwarfs the sub-100 ns round-trip of real CXL hardware [33], [34]. Furthermore, interception is only feasible at page granularity (4 KB or larger)—cache-line trapping would require thousands of context switches per microsecond. Real CXL hardware operates at 64-byte granularity, so any purely software-trap approach cannot replicate fine-grained sharing semantics [35]. Applications also assume memory latency, atomicity, and ordering guarantees that trap-based emulators routinely violate, breaking transparent compatibility with unmodified workloads.

Cache coherence. Hardware CXL 3.0 issues snoop requests and invalidations in silicon with nanosecond timing. A software emulator must replicate this explicitly: every remote write needs a `clflush/mfence` sweep of all stale cached copies—slow, error-prone, and requiring careful serialization.

Scalability. Each additional host multiplies coherence traffic: a shared write can trigger snoop messages to all other hosts, and every topology change requires Fabric Manager coordination across all participants. Naive broadcast-snoop or centralized-lock designs may experience performance degradation, making it difficult to reach the 16-host MH-SLD specification limit.

OCEAN addresses each challenge above as follows. Device-level interception—not page-fault traps—achieves cache-line granularity with near-native throughput. A dedicated software coherence layer (§IV-C2) replicates MESI protocol semantics with distributed snoop filters. Active latency injection (§IV-D1) enforces simulated timing per access rather than computing it offline. A token-bucket rate limiter (§IV-D2) enforces per-device bandwidth ceilings. The Fabric Manager centralizes topology and allocation, simplifying multi-host coordination. Integration with the upstream Linux CXL driver stack ensures unmodified applications run as-is [2].

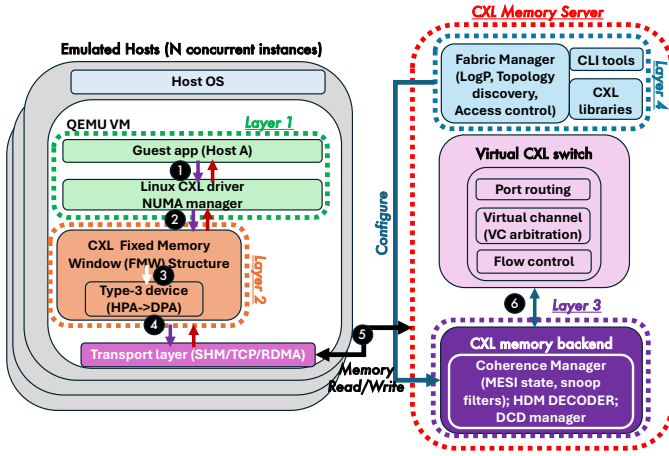


Fig. 1: Architecture of OCEAN emulator.

IV. OCEAN SYSTEM DESIGN

This section presents the architecture and internal design of OCEAN. We adopt a top-down exposition: Section IV-A introduces the layered architecture and two operating modes; Section IV-B describes the CXL device and fabric switch models; Section IV-C details the coherence engine and the pluggable transport layer; and Section IV-D explains the performance-modeling subsystem that injects realistic timing into every emulated memory access.

A. Architecture Overview

Figure 1 shows the four-layer organization of OCEAN. Layer 1, the **Hardware Abstraction Layer** leverages Kernel-based Virtual Machine (KVM) and Virtual Function I/O (VFIO) inside QEMU for efficient device passthrough to present each virtual machine as a CXL-capable host, complete with PCIe root complexes and CXL root ports. Layer 2, the **CXL Device Emulation Layer** instantiates Type-3 memory expanders and CXL fabric switches, each configurable via the Newick tree topology notation. The **Multi-Host Coordination Layer** houses the distributed memory server, the coherence engine, and the pluggable transport subsystem; this layer is the primary contribution of OCEAN over single-host CXL emulators. Finally, the **Management and API Layer** provides the Fabric Manager daemon, a command-line interface for topology and capacity control, and user-space libraries that expose CXL memory through standard Linux interfaces (numactl, DAX, mmap).

OCEAN supports two complementary operating modes. In *single-node* mode, the OCEAN core intercepts hardware performance-monitoring counters (PMCs) [36], [37] on the host CPU to compute per-access CXL latencies without requiring a network round-trip [38], [39]; this mode is well suited for rapid functional testing and lightweight profiling of NUMA-aware workloads on a single machine. In *multi-host* mode, two or more QEMU instances connect to a centralized OCEAN memory server over shared memory (SHM), TCP, or RDMA; each memory access traverses the

full emulated path—device model, switch, coherence engine, and transport—enabling large-scale experiments with up to 16 hosts, the architectural maximum defined by the CXL 3.0 MH-SLD sub-specification [5]. This paper focuses primarily on the multi-host mode, which exercises all layers of the emulation stack.

The numbered steps in Figure 1 trace the critical path of a memory access in multi-host mode. ① A guest application issues a load or store instruction targeting a CXL-backed virtual address. ② The Linux CXL driver, via the kernel NUMA manager, maps the virtual address through the CXL Fixed Memory Window (FMW) to a host physical address (HPA) in the device’s decode range. ③ The QEMU Type-3 device model intercepts the Memory-mapped IO (MMIO) access, translates HPA to Device physical address (DPA) using the programmed Host-managed device memory (HDM) decoders, and constructs a CXL.mem request. ④ The request is forwarded over the selected transport to the OCEAN memory server. ⑤⑥ The server consults the coherence engine, which may issue snoop or invalidation messages to other hosts. The server computes the simulated access time T_{sim} (incorporating switch hops, coherence cost, and contention) and returns the data. The device model then injects T_{stall} as a vCPU stall before retiring the instruction (Section IV-D1).

B. CXL Device and Fabric Emulation

The CXL Device Emulation Layer (Layer 2 in Figure 1) instantiates three classes of components: Type-3 memory devices, virtual CXL switches, and the Fabric Manager. We describe each component by first establishing its role and capabilities, then detailing the internal mechanisms that realize those capabilities.

1) *Type-3 Memory Device Model*: The Type-3 memory device is the fundamental building block of CXL memory expansion. Each emulated device in OCEAN provides the following key capabilities. (1) *Flexible address mapping*: up to 32 *HDM decoders* enable fine-grained partitioning of the device’s physical capacity across hosts, each mapping a contiguous host physical address range to device-local memory with configurable interleave granularity and a per-host target list. (2) *Dynamic capacity management*: Dynamic Capacity Device (DCD) extends the static HDM model with runtime capacity management. The Fabric Manager can hot-add or hot-remove memory extents in 256 MB granules, and the device notifies the guest OS so it can map or unmap the affected region. (3) *Multi-host access control*, supporting up to 16 concurrent host connections with individually configurable access permissions. In addition, performance monitoring counters expose bandwidth utilization, access latency, and error telemetry.

To realize these functionalities, a CXL-compliant *mailbox* serves as the management channel through which the host issues commands—HDM decoder programming, DCD extent operations, firmware queries, and event log retrieval—with realistic command-completion timing to prevent guest drivers from observing artificially fast responses. A 256-KB *Label*

Storage Area (LSA) provides persistent, host-accessible meta-data storage for device configuration and namespace labels. We provide the implementation details in Section V-A.

2) *CXL Switch*: The second component instantiated by layer 2, *virtual CXL switch*, supports up to 256 ports with hardware routing tables in unicast, multicast, and anycast modes, three QoS traffic classes with weighted round-robin (WRR) arbitration, and both in-band and out-of-band management interfaces. The CXL switch connects hosts to devices through a configurable tree topology. Beyond basic packet forwarding, faithful switch emulation requires modeling the per-port state machine, virtual channel (VC) arbitration, and credit-based flow control defined by the CXL 3.0 specification.

Port state machine. Each of the 256 virtual ports follows a five-state lifecycle: RESET → TRAINING → CONFIGURED → ACTIVE → DISABLED, with a parallel ERROR sink reachable from any active state. During TRAINING, the port exchanges link-capability Transaction Layer Packets (TLPs) and negotiates the number of enabled lanes and link speed. OCEAN models this exchange with a configurable training latency (default 2 ms, matching published bring-up measurements) before transitioning to CONFIGURED. An ACTIVE port that stops receiving heartbeat flits within a configurable timeout (default 100 ms) transitions to DISABLED and notifies the Fabric Manager via an out-of-band interrupt, triggering rerouting.

Virtual channel arbitration. The CXL 3.0 specification defines three virtual channels: VC0 for non-coherent memory transactions, VC1 for coherence messages (Snp*, Rsp*), and VC2 for management and configuration traffic. OCEAN implements weighted round-robin (WRR) arbitration across the three channels at each egress port, with default weights 4:2:1 (VC0:VC1:VC2), matching the throughput priority recommendations in the specification. Weights are runtime-configurable via the Fabric Manager CLI.

Credit-based flow control. Real CXL links use credit-based flow control to prevent egress buffer overflow. Each virtual port in OCEAN maintains separate credit pools for request-class and response-class flits per VC. Before forwarding a flit, the switch deducts the appropriate credits from the downstream port’s pool; credits are returned once the downstream port acknowledges receipt. The credit pool sizes are configurable at instantiation time (default 32 request credits and 16 response credits per VC per port) to model different hardware.

Routing. The switch supports unicast, multicast (delivering to a group of ports simultaneously), and anycast (load-balancing across redundant paths) routing modes as defined by CXL 3.0. Every switch traversal incurs a configurable per-hop latency (default 65 ns), and the switch models congestion-induced queuing delay by tracking in-flight requests. Section V-A details the forwarding table implementation.

3) *Fabric Manager*: The third component, the *Fabric Manager* (FM), is the sole centralized control-plane and performs automatic topology discovery at boot, manages global-to-device address mapping, implements error detection with component isolation, and enforces per-host access control.

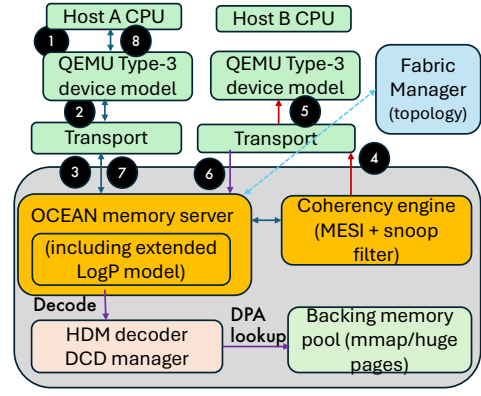


Fig. 2: OCEAN distributed memory server. An example workflow of cross-host coherent read.

Topology discovery. At boot, it executes a four-state discovery protocol finite state machine (FSM) per device: UNKNOWN → DISCOVERED → CONFIGURED → ONLINE. In UNKNOWN, the FM probes each potential device address in the fabric with an IDENTIFY command and a 500-ms timeout. A valid response promotes the device to DISCOVERED, where the FM reads its capability registers (device type, HDM decoder count, DCD support) and assigns a globally unique Fabric Port ID. In CONFIGURED, the FM programs HDM decoders to partition device physical capacity across requesting hosts, installs routing entries in intervening switches, and distributes the global address map to all host drivers. Once all decoders and routes are committed, the device transitions to ONLINE and begins serving requests.

Global address space allocation. The FM manages global memory as a sorted interval tree of (gpa_start, gpa_end, owner_host, device_fpid) records, implemented using the kernel’s rb_tree API in the FM daemon’s address-space manager. Allocation requests from guest drivers specify a size and an optional NUMA-locality hint (preferred host ID); the FM performs a first-fit search over free intervals on the hinted device, falling back to global first-fit if the preferred device lacks capacity. The interval tree supports $O(\log n)$ allocation and lookup, keeping FM response latency below 20 μ s even with thousands of active allocations. Freed extents are coalesced with adjacent free intervals to prevent fragmentation, using the same logic as the Linux kernel’s gen_pool allocator.

C. Coherency Engine and Transport

CXL 3.0 extends PCIe with hardware-managed cache coherency across hosts sharing a common memory pool. OCEAN replicates this functionality through a centralized coherency engine and a modular transport layer that decouples protocol logic from the physical interconnect.

1) *Directory-Based MESI Protocol*: OCEAN implements the full CXL 3.0 coherency protocol including the complete MESI state set and CXL-specific extensions (Figure 2 illustrates a cross-host coherent read). The coherency engine maintains per-cacheline state using the four-state MESI pro-

tocol: **Modified**, **Exclusive**, **Shared**, and **Invalid**. Each 64-byte cacheline is tracked by a directory entry recording the current state, the owning host ID, and a sharer bitmap. Directory entries are protected by fine-grained per-cacheline locks to allow concurrent operations on disjoint addresses.

On a read, the engine performs a directory lookup. If the requesting host holds the line in **Invalid** state, the engine checks whether another host holds it in **Modified** state; if so, it issues a snoop, which either supplies data directly (transitioning from **Modified**) or downgrades to **Shared**. The requester receives the line in **Shared** or **Exclusive** state depending on whether other sharers exist. On a write, the engine issues `SnpInv` messages to all sharers, collects acknowledgments, and transitions the writer to **Modified**.

A *back-invalidation queue* tracks dirty updates across concurrent client threads. When the server detects that a cacheline has been written by one host, it enqueues invalidation entries for all other hosts holding stale copies; remote hosts process these lazily on their next access, amortizing cross-host coherency overhead.

Distributed snoop filters at each host and switch limit broadcast snoops to relevant nodes, and direct peer-to-peer transfers via the CXL switch allow one host to service a cache miss from another host's cache without indirecting through main memory.

2) *Software Cache Coherence (SCC)*: For memory regions that use software-managed coherence—persistent structures or inter-host communication buffers where hardware coherence is not guaranteed—OCEAN provides an SCC layer with a per-region coherence mode selector. In SCC mode, the CXL device model instruments every write to issue `clflush/mfence` against all written cache lines before the write is considered complete. The Fabric Manager maintains a per-region invalidation bitmap: when a host writes a cache line in SCC mode, the FM distributes invalidation tokens to all other hosts holding that line in a **Shared** or **Exclusive** state. Remote hosts honor these tokens lazily on the next access.

3) *Synchronization Primitives*: OCEAN exposes global atomic operations (compare-and-swap, fetch-and-add) visible to all hosts, a hardware barrier that releases all participants only after every host has arrived, and a distributed lock mechanism in the FM with configurable FIFO or priority-based fairness.

4) *Transport Layer*: OCEAN decouples CXL protocol logic from the underlying transport through a modular pluggable transport layer with three backends. The **SHM** (single-node) backend, POSIX shared memory, provides a zero-copy data path via circular ring buffers, giving $\approx 1\mu\text{s}$ round-trip latency and making this mode ideal for functional verification. The **TCP/IP** (multi-node, compatibility) backend serializes CXL TLPs into TCP streams. Although TCP involves higher latency, but universal deployability and its applicability for large-scale topology tests where timing fidelity is secondary, makes it suitable for functional correctness test. With the **RDMA** (multi-node, performance) backend, the `ibverbs` API maps CXL load/store to RDMA `READ/WRITE`, by-

Algorithm 1 Active Latency Injection with LogP-Calibrated Delay

```

1: procedure COMPUTELATENCY( $src, dst, sz, \rho$ )
2:    $hops \leftarrow \text{TOPOLOGYDISTANCE}(src, dst)$ 
3:    $L \leftarrow hops \cdot L_{\text{switch}}$   $\triangleright$  link + switch depth
4:    $o_s \leftarrow o_s^{\text{cal}}(sz)$   $\triangleright$  calibrated sender overhead
5:    $o_r \leftarrow o_r^{\text{cal}}(sz)$   $\triangleright$  calibrated receiver overhead
6:    $g \leftarrow sz / BW_{\text{link}}$   $\triangleright$  gap (serialization delay)
7:    $L_{\text{base}} \leftarrow o_s + L + o_r + g$ 
8:    $T_{\text{sim}} \leftarrow L_{\text{base}} \cdot C(\rho)$   $\triangleright$  contention scaling factor
9:   return  $T_{\text{sim}}$ 
10: end procedure

11: procedure INJECTLATENCY( $t_{\text{start}}, L_{\text{sim}}$ )
12:   if  $T_{\text{sim}} < T_{\text{min}}$  then return
13:   end if
14:    $t_{\text{now}} \leftarrow \text{CLOCKMONOTONIC}$ 
15:    $T_{\text{IPC}} \leftarrow t_{\text{now}} - t_{\text{start}}$ 
16:   if  $T_{\text{IPC}} \geq T_{\text{sim}}$  then return
17:   end if
18:    $R \leftarrow L_{\text{sim}} - T_{\text{IPC}}$ 
19:   if  $R > 10\mu\text{s}$  then
20:      $\text{NANOSLEEP}(R - 1\mu\text{s})$ 
21:   end if
22:   repeat
23:      $t_{\text{now}} \leftarrow \text{CLOCKMONOTONIC}$ 
24:   until  $t_{\text{now}} - t_{\text{start}} \geq T_{\text{sim}}$ 
25: end procedure

```

passing the remote CPU. Coherency messages use RDMA `SEND/RECV`. The RDMA transport sustains near line-rate bandwidth across physical servers and is the backend used for all performance evaluations.

D. Performance Modeling

OCEAN goes beyond functional emulation by actively injecting realistic timing into every memory access. The performance modeling subsystem comprises three components.

1) *Active Latency Injection*: Rather than computing latency offline from trace data (counters), OCEAN stalls each vCPU for the exact duration of the simulated CXL access, making timing visible to the guest's instruction stream in real time. Because the IPC round-trip to the memory server already consumes wall-clock time, naive addition of the full simulated latency would overcount. OCEAN compensates:

$$T_{\text{stall}} = \max(0, T_{\text{sim}} - T_{\text{IPC}}) \quad (1)$$

where T_{sim} is the simulated access time returned by the server and T_{IPC} is the measured IPC round-trip. Algorithm 1 shows the implementation. For delays above $10\mu\text{s}$, `nanosleep` handles the bulk and a spin-wait covers the final microsecond; shorter delays spin-wait entirely since `nanosleep` has $\approx 50\mu\text{s}$ minimum granularity on Linux.

Injection is wired into `cxl_type3_read` and `cxl_type3_write` in QEMU's CXL device model:

Listing 1: Active latency injection in `cxl_type3_read`.

```

1 struct timespec ts;
2 uint64_t start_ns = 0;
3 if (g_memsim.latency_inject) {
4     clock_gettime(CLOCK_MONOTONIC, &ts);
5     start_ns = ts.tv_sec * 1e9 + ts.tv_nsec;
6 }
7 cxl_memsim_request(READ, dpa, size, &resp);
8 memcpy(data, resp.data, size);
9 cxl_memsim_inject_latency(start_ns, resp.latency_ns);

```

Because `cxl_type3_read/write` execute in the vCPU thread, stalling them is semantically correct: the vCPU cannot retire the memory instruction until the simulated access completes. Injection is enabled via `CXL_LATENCY_INJECT=1`; accesses below `CXL_LATENCY_INJECT_MIN_NS` (default 80 ns) are skipped.

2) *Bandwidth Constraint Enforcement*: Latency injection alone does not prevent aggregate bandwidth that exceeds a real CXL link’s rated capacity under high concurrency. OCEAN therefore enforces per-device bandwidth ceilings by adding a token-bucket rate limiter: each emulated device maintains a bucket of capacity C bytes refilling at R GB/s. Before servicing a request of size s , the memory server waits until s tokens are available; the resulting response delay is then fed back into Equation 1, extending the vCPU stall. Tokens refill at 64-byte cache-line granularity to match flit-level CXL scheduling. Both R and C are configurable at runtime via the Fabric Manager CLI, enabling link-speed sweep experiments without rebuilding the emulator. Combined with active latency injection, this mechanism models the full CXL performance envelope: steady-state saturation and bursty patterns.

3) *Extended LogP Latency Model*: OCEAN models the simulated latency L_{sim} for each cross-host access by extending the LogP framework [40]. The standard LogP single-message time $T = o_s + L + o_r$ is augmented by decomposing the network latency L into three CXL-specific sub-components. For a single memory access, the modeled one-way access time:

$$T_{\text{sim}} = o_s + n_{\text{hops}} \cdot L_{\text{switch}} + L_{\text{coherency}} + L_{\text{congestion}} + o_r$$

where o_s and o_r are sender and receiver processor overheads, n_{hops} is the number of switch traversals, and $n_{\text{hops}} \cdot L_{\text{switch}}$ is the cumulative per-switch propagation delay. $L_{\text{coherency}}$ accounts for snoop and invalidation round-trips (zero for lines already in Exclusive or Modified state at the requester). $L_{\text{congestion}}$ is derived from current link utilization via a piecewise-linear contention model. These parameters are calibrated against real CXL hardware using an MPI-based benchmark (Section VI-D).

Combined, active latency injection, bandwidth enforcement, and extended LogP model, the full CXL performance envelope, from single-access latency through sustained-bandwidth saturation to bursty contention under multi-host sharing.

V. IMPLEMENTATION

Translating the design of §IV into a working system requires solving a sequence of engineering problems at increasing levels of the stack. First, the QEMU device model must faithfully

reproduce the CXL 3.0 register-level interface so that unmodified guest drivers can discover and program the emulated hardware (Section V-A). Second, the guest kernel must integrate that hardware into Linux’s memory subsystem—NUMA nodes, DAX regions, cache-flush paths—without exposing the emulation boundary to applications (Section V-B). Third, the transport layer must bridge the gap between the QEMU device model and the memory server with minimal overhead, since every microsecond of IPC cost tightens the latency budget available for faithful timing simulation (Section V-C). We describe each in turn.

A. QEMU Device Model Internals

The CXL 3.0 specification defines precise register layouts and command protocols that guest drivers rely on for device discovery, decoder programming, and capacity management. To reproduce these interfaces in OCEAN, we modified QEMU 10.0 with approximately 15,000 lines of new C code. We discuss these modifications below.

Mailbox command processing. We implement the mailbox (Section IV-B1) as a 512-byte shared register window in the device’s BAR2 region, following the CXL 3.0 register layout. A guest write to the `MBOX_CTRL` register atomically transfers ownership of the mailbox to the device thread, which processes the command synchronously, writes the response payload and completion code, and sets the `Ready` bit to signal completion. To model real hardware latency, the device thread sleeps for a configurable per-command latency (default 50 μ s for HDM decoder updates, 200 μ s for DCD extent operations) before signaling, preventing drivers that busy-poll the mailbox from observing instantaneous responses.

HDM decoder programming sequence. Each HDM decoder is a 128-bit register pair (`HDM_DECODER_BASE`, `HDM_DECODER_SIZE`) plus a 64-bit `HDM_DECODER_CTRL` that enables the decoder and specifies the interleave granularity and target device list. Our implementation tracks decoder state in a per-device array and re-evaluates the address-decode table on every write to any decoder register. Guest accesses that fall in a decoded range are forwarded directly to the backing memory region; accesses outside all active decoders raise a UR (Unsupported Request) completion, matching the hardware behavior that the Linux CXL driver’s error-path tests rely on.

DCD event notification. When the FM instructs OCEAN to add or remove a DCD extent, the device model must notify the guest driver via a Dynamic Capacity Event (DCE) record in the event log. We implement the event log as a 16-entry circular buffer in the device’s mailbox payload region. Adding a new extent appends a DCE record with the physical address, size, and event type, then raises a legacy PCI INTx interrupt to the guest. The Linux DCD driver reads the event log via a `GET_EVENT_RECORDS` mailbox command, processes each record, and acknowledges completion with `CLEAR_EVENT_RECORDS`, at which point the device model dequeues the processed entries. This end-to-end notification path is exercised by our hot-plug fidelity tests and must round-trip in under 5 ms to pass.

Switch routing table management. The virtual CXL switch maintains a forwarding table mapping each 8-bit Device Port ID (DPID) to an egress port index. The table is populated during fabric discovery by the FM and is updated atomically via a reader-writer lock to avoid corrupting in-flight transactions. For multicast, the table maps a multicast group ID to a bitmask of egress ports; the switch clones the flit and enqueues one copy per set bit. For anycast, the switch selects the egress port with the shortest observed queue depth at the time of forwarding, load balancing across redundant paths.

B. OS Kernel Modules

The device model described above presents a CXL-compliant hardware interface to the guest; however, the guest kernel must still discover that interface, integrate the exposed memory into Linux’s NUMA and memory-management subsystems, and execute cache-flush sequences on behalf of the coherency engine. We implemented three kernel modules for Linux 6.16-rc7 running in guest VMs, each addressing a distinct aspect of CXL guest integration to bridge this gap.

CXL driver extension: device probe and NUMA registration. The extended driver hooks into the existing `cxl_pci` probe path. On device probe, it reads the device’s CXL DVSEC (Designated Vendor Specific Extended Capability) registers to determine whether the device is a Type-3 memory expander with GFAM or MH-SLD capabilities. For each GFAM region, the driver calls `memory_hotplug_lock` and invokes `add_memory_driver_managed` with the global CXL physical address range and the `MEMHP_MERGE_RESOURCE` flag to prevent the kernel’s resource tree from splitting the range. A corresponding NUMA node is created via `register_memory_node_under_compute_node` with a synthetic NUMA distance table that reflects the emulated CXL hop count, allowing `numactl --membind` and `libnuma` to work without modification.

DCD lifecycle: extent probe, DAX region, and detach. DCD extents follow a three-phase lifecycle managed by the memory management module. In the *probe* phase, the driver receives a DCE interrupt, reads the event log, and calls `devm_memremap_pages` to map the new extent into the kernel’s struct-page array, making it allocatable by the buddy allocator. In the *active* phase, the extent is exposed as a DAX character device (`/dev/daxN.M`) that applications can `mmap` for direct byte-addressable access, or as a conventional memory region via the `kmem` driver. In the *detach* phase, triggered by a removal DCE, the module calls `offline_and_remove_memory` to migrate all live pages off the extent before signaling the Fabric Manager that the extent is safe to reclaim. The detach path is the most complex: it must handle the case where pages are pinned (by DMA or `get_user_pages`), in which case it retries the offline with exponential backoff up to a 500 ms timeout before returning an error to the FM.

Coherency module: cache-range flush integration. The coherency module registers a notifier with the QEMU

device model through a shared memory control word in the SHM transport’s ring buffer header. When the device model sets the `INVAL_REQUEST` bit for a given physical address range, the module’s IRQ handler calls `cacheflush_on_cache_line_size_aligned_range`, iterating over the range in 64-byte steps and issuing `clflush_opt` on each line. A subsequent `mfence` serializes the flushes before the module clears the `INVAL_REQUEST` bit and signals completion. On processors that support `CLDEMOTE`, the module preferentially uses demotion rather than full flush for SCC-mode read-sharing scenarios, avoiding unnecessary DRAM writeback traffic.

C. Transport Layer: RDMA Connection Management

Because the RDMA transport is the primary backend for performance evaluation, we describe its connection setup and queue-pair (QP) management in detail.

Each QEMU instance (emulated CXL host) establishes a single reliable connected (RC) QP to the memory server at startup. QP creation follows the standard `ibverbs` flow: `ibv_create_qp` with a send and receive completion queue of depth 256, followed by a `INIT` → `RTR` → `RTS` transition exchanging the remote QP number, LID, and PSN via an out-of-band TCP control channel. Memory regions backing the emulated CXL device are registered with `ibv_reg_mr` using `IBV_ACCESS_REMOTE_READ` | `IBV_ACCESS_REMOTE_WRITE` | `IBV_ACCESS_LOCAL_WRITE`, and the resulting `rkey` is distributed to all QEMU instances by the Fabric Manager at fabric-initialization time.

For read requests, the QEMU device model posts an RDMA READ work request specifying the remote virtual address (the DPA translated to the memory server’s virtual address space) and the `rkey`; the response is placed directly into the guest’s DMA bounce buffer, avoiding a software copy. For coherency messages that require ordered delivery, the transport uses RDMA SEND on a separate coherency QP with immediate data encoding the coherency opcode (snoop, invalidate, flush-ack), ensuring that the completion order matches the order in which the messages were generated. The RDMA backend sustains 196 Gb/s of aggregate throughput across four concurrent QEMU instances before hitting the NIC’s PCIe bandwidth limit, leaving adequate headroom for the emulated CXL 3.0 device’s 64 GT/s link.

VI. EVALUATION

A. Experimental Setup

Hardware testbed. To ground our emulation results against real CXL hardware – we use a two-node CXL memory system in which each node connects to an FPGA-based memory accelerator via a CXL 1.x-to-2.x protocol conversion interface over PCIe 5.0. Each compute node contains a dual-socket Intel Xeon Gold 5420+ (28 cores/socket, 4.1 GHz) with 48 KB L1d, 2 MB L2, and 52.5 MB shared L3 per core, backed by 1 TB local DRAM. The FAM device provides 32 GB of CXL-attached memory managed by a memory server daemon.

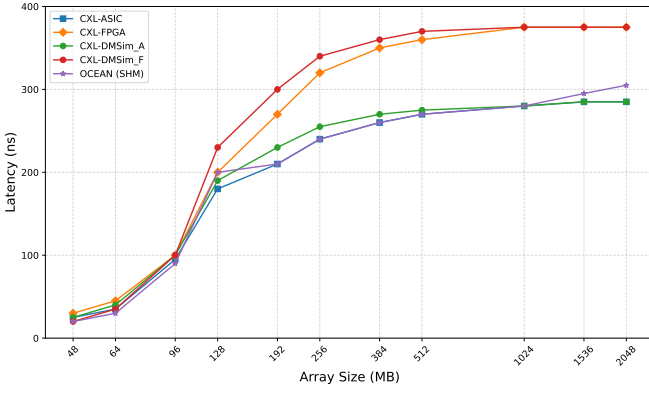


Fig. 3: **LMbench**: Measured random memory access latency reported by OCEAN and CXL-DMSim

Benchmarks. We use STREAM [41] for sustained memory bandwidth (Copy, Scale, Add, Triad kernels), LMbench [42] for single-host memory access latency across working-set sizes (`lat_mem_rd`, 64-byte stride), and the OSU Micro-Benchmarks [43] for multi-host collective communication latency (MPI Allgather). For key-value workloads we use YCSB [44], which provides configurable read/write ratios and read-modify-write (RMW) operations. For transactional database workloads we use TPC-C, the industry-standard OLTP benchmark modeling order processing; we focus on NewOrder (order creation) and Payment (balance update), the most write-intensive and contention-sensitive transaction types. All database experiments use Two-Phase Locking (2PL) concurrency control, in which each transaction acquires all locks before committing, making throughput directly sensitive to cross-host lock acquisition and cache-line invalidation latency. For scientific simulation we use GROMACS [45] with the PEPSIN protein system.

B. Fidelity of OCEAN

We evaluate OCEAN’s emulation accuracy using three standard microbenchmarks that isolate distinct performance dimensions. Together, these establish OCEAN’s baseline fidelity before full-application case studies.

Memory Latency. Before demonstrating OCEAN’s *multi-host* emulation capability, we first establish its *single-host* performance by evaluating its memory access latency using the `lat_mem_rd` microbenchmark from LMbench [42], configured with a 64-byte stride and array sizes up to 2048 MB, ensuring full coverage from cache-resident to memory-bound regimes. To validate OCEAN’s comparable performance against CXL-DMSim [2], a single-host CXL simulator, we include the real CXL device (ASIC and FPGA) and simulation latencies reported in their work. Figure 3 shows all configurations exhibit the expected trend of increasing latency with growing working set size. At large array sizes (≥ 512 MB), OCEAN (SHM) closely tracks the reported CXL-ASIC latency just like the simulated ASIC (CXL-DMSim_A) latency.

Bandwidth. Figure 4 presents STREAM benchmark [41] results to demonstrate OCEAN’s multi-host CXL emulation

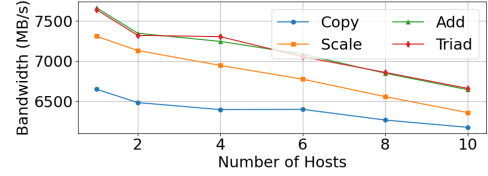


Fig. 4: STREAM benchmark bandwidth reported by OCEAN for Copy, Scale, Add, and Triad kernels across varying host counts.

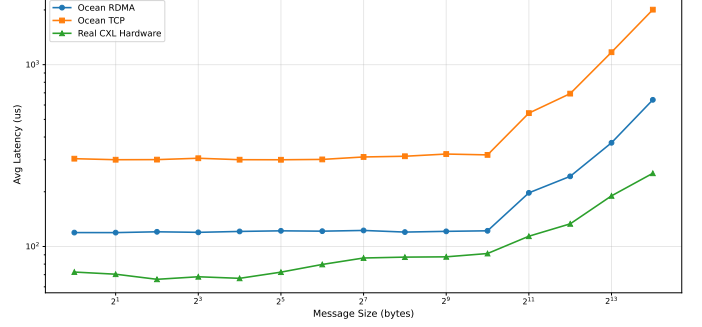


Fig. 5: OSU MPI Allgather latency vs. message size: OCEAN (RDMA and TCP backends) validated against real CXL hardware.

capability. The figure reports per-host memory bandwidth (MB/s) for the four STREAM kernels (Copy, Scale, Add, Triad) as the number of emulated hosts sharing a CXL memory pool increases from 1 to 10. All four kernels exhibit a gradual decline in per-host bandwidth with increasing host count, consistent with the expected contention behavior: as more hosts compete for access to the shared CXL memory pool, increased arbitration overhead at the CXL switch/fabric raises effective latency and reduces the bandwidth available to each individual host.

OSU benchmark. Figure 5 presents average latency results for the OSU MPI Allgather benchmark [43], validating OCEAN’s multi-host CXL emulation capability against a real CXL hardware baseline. The benchmark exercises collective communication across multiple emulated hosts sharing a CXL memory pool, directly stressing the inter-host coordination that OCEAN is designed to emulate. For small messages (up to 2^{10} bytes), latency remains relatively flat across all configurations, dominated by fixed protocol overhead. OCEAN with RDMA consistently tracks the real hardware curve within approximately $2\times$, while the TCP backend incurs roughly $3\times$ higher latency due to kernel networking stack overhead. Beyond 2^{10} bytes, all three configurations exhibit increasing latency as data transfer costs dominate, with RDMA maintaining closer fidelity to real CXL hardware than TCP. These results validate OCEAN’s ability to faithfully emulate multi-host CXL memory access patterns, with RDMA offering the most representative transport backend [46].

C. Usecases

1) *Exploring Hardware Cache Coherence with OCEAN*: CXL 3.0 introduces optional hardware-managed cache coherence (HCC) across hosts sharing a common memory pool — a capability that no prior emulator can exercise because it

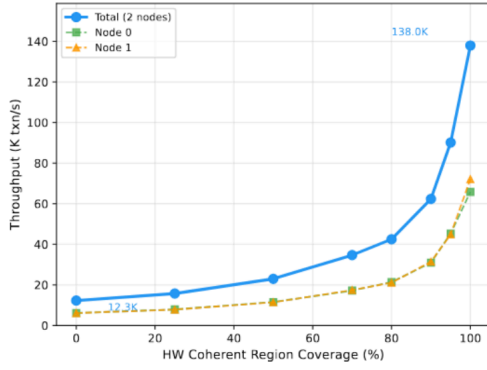


Fig. 6: Per-node TPC-C NewOrder throughput vs. HCC coverage (2PL, 2 nodes). Software-flush elimination drives throughput from 12.3K to 138K txn/s.

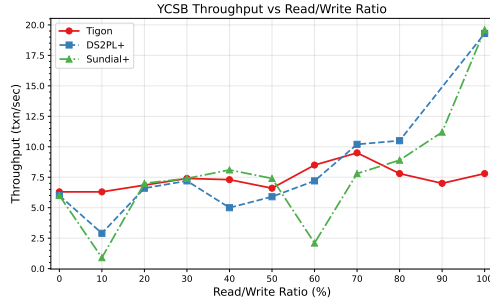


Fig. 7: YCSB throughput vs. read/write ratio (2 nodes). Tigon (hardware coherence) is stable; DS2PL+ and Sundial+ (software coherence) degrade beyond 40% writes.

requires multi-host fabric support. OCEAN enables, for the first time, controlled experiments that isolate the effect of HCC on real database workloads by allowing the operator to vary the fraction of shared memory regions that use hardware coherence (with the remainder falling back to software-managed coherence via explicit `tcflush/mfence` sequences).

Figure 6 illustrates the per-node throughput of TPC-C NewOrder — a transaction that creates a new customer order by reading and writing across warehouse, district, and order tables — as HCC coverage increases from 0% to 100% on a two-node deployment under Two-Phase Locking (2PL) concurrency control. At 0% HCC (fully software-managed coherence), aggregate throughput across both nodes is limited

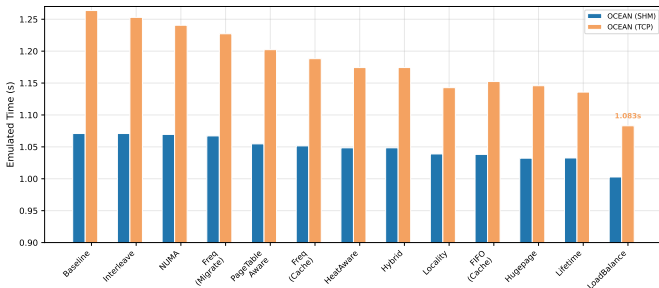


Fig. 8: GROMACS PEPSIN execution time across twelve CXL memory-placement policies. Policy choice yields up to 7% performance variation.

to 12.3K transactions per second: every cross-host write forces a full cache-line flush and memory fence round-trip through the CXL link, serializing what should be concurrent operations. As HCC coverage rises, the hardware coherence protocol replaces these expensive software flushes with silicon-speed snoop and invalidation messages. At 100% HCC, throughput reaches 138K transactions per second — an $11\times$ improvement — confirming that the software-flush overhead, not memory bandwidth or lock contention, is the dominant bottleneck in cross-host transactional workloads. This finding, observable only on a multi-host coherent platform like OCEAN, quantifies for the first time the performance budget that CXL 3.0 hardware coherence unlocks for database systems.

Figure 7 complements this result by holding the coherence mechanism fixed and varying the workload’s read/write ratio. Three concurrency protocols are compared on the YCSB key-value benchmark across two OCEAN nodes: Tigon, which routes all shared accesses through CXL hardware coherence; and DS2PL+ and Sundial+, which extend distributed strict two-phase locking and optimistic concurrency control, respectively, with CXL-aware software cache management. Tigon delivers stable throughput across the entire spectrum from read-only to write-only workloads, because the hardware coherence path handles invalidations at constant cost regardless of write frequency. DS2PL+ and Sundial+, by contrast, degrade sharply once the write ratio exceeds approximately 40%: each write triggers a `clflush` whose latency on CXL-attached memory is $3\text{--}5\times$ higher than on local DRAM, and this cost compounds multiplicatively under contention. At read-dominated ratios, all three protocols converge, because reads in Shared state require no invalidation traffic. This crossover pattern — software coherence viable for read-heavy workloads, hardware coherence essential for write-heavy ones — matches trends observed on real CXL hardware and provides system designers with a concrete decision boundary for coherence-mode selection.

2) *Design-space exploration*: Beyond validating fidelity against real hardware, OCEAN serves as a platform for exploring memory-management policies that will be critical to extracting performance from CXL 3.0 fabrics. Figure 8 illustrates this capability by sweeping twelve memory-management and page-placement policies on a GROMACS PEPSIN workload that a CXL fabric manager or OS memory tiering layer could implement: Baseline (default allocation), Interleave (round-robin across NUMA nodes), NUMA-aware placement, frequency-based migration, page-table-aware caching, heat-aware tiering, hybrid policies, locality-optimized placement, FIFO caching, huge-page promotion, lifetime-based allocation, and load balancing. Each policy represents a distinct approach to managing data across local DRAM and CXL-attached memory — ranging from static policies (NUMA-aware binding, interleaving, huge-page promotion) to dynamic ones (frequency-based migration, heat-aware tiering, lifetime-based allocation, load balancing). OCEAN with SHM consistently outperforms the TCP baseline across all policies, and more importantly, reveals that policy choice alone accounts for up to a 7%

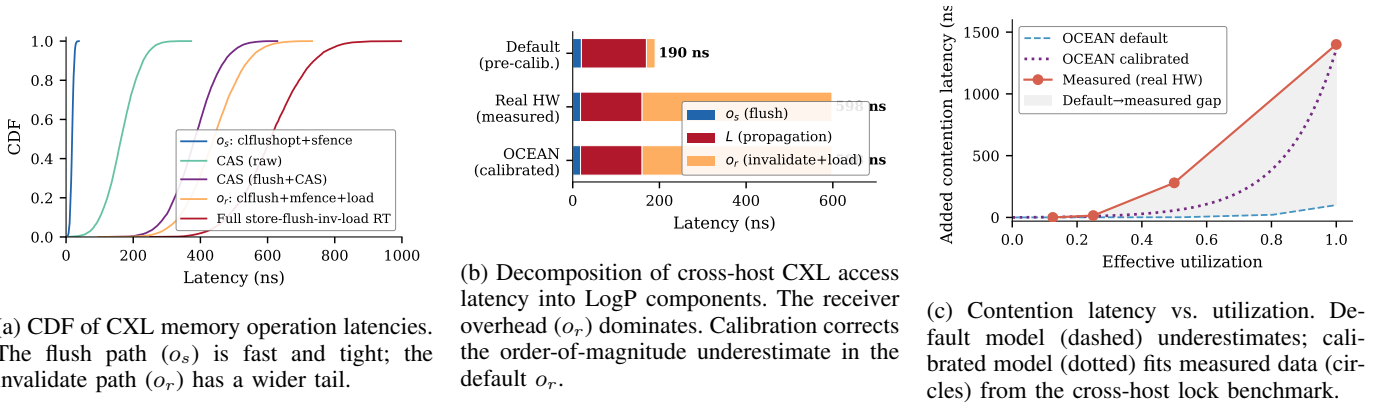


Fig. 9: LogP latency model characterization and calibration for cross-host CXL access.

performance spread (LoadBalance vs. Baseline) — a gap that would compound at scale. Such sweeps are infeasible on scarce real CXL hardware and invisible to single-host emulators. OCEAN enables researchers to identify optimal placement strategies before hardware is available [47], [48].

D. LogP Model Calibration

OCEAN models inter-host CXL communication using the LogP framework [40] (Section IV-D3). Accurate estimation of the parameter values of LogP is essential for timing fidelity; we therefore developed an MPI-based calibration benchmark (`cxl_switch_lock_bench_mpi`) that runs on two hosts sharing CXL Type-3 memory through a CXL switch and measure each parameter independently.

Methodology. Each parameter maps to a distinct cache-line operation. The first parameter $o_s = \text{clflushopt} + \text{sfence}$: the sender evicts a dirty cache line to the CXL device (`clflushopt` initiates a write-back with cache-line invalidation; `sfence` ensures the write-back is globally visible before proceeding). The second one, $o_r = \text{clflush} + \text{mfence} + \text{load}$: the receiver discards any stale cached copy (`clflush` invalidates the line), enforces ordering (`mfence` prevents the subsequent load from executing speculatively before invalidation completes), and fetches the current value directly from the CXL device. Third, L is derived from a cross-host ping-pong round-trip, where $L = (\text{RTT} - 2o_s - 2o_r)/2$; as the calibration path traverses two switch hops, $L_{\text{hop}} = L/2$. g is the per-line cost under 512-line streaming flushes. Both MPI ranks map the same DAX device; MPI barriers synchronize phases while measurements use only direct CXL memory access.

Figure 9b shows the decomposition. The dominant cost is o_r (invalidate + reload), accounting for over 70% of the point-to-point latency. The default $o_r = 20$ ns underestimates this by 22 $\times$; after calibration, the emulated RTT matches real hardware within 5%. Table II lists the fitted values.

Contention model fitting. OCEAN’s `FabricLink` models congestion via a piecewise-linear function of link utilization. To calibrate this, both hosts contend on N shared CAS locks ($N = 1, 2, 4, 8$); lower N means higher effective utilization. Figure 9c shows that the default model

Param.	Default	Calibrated	Unit	Method
o_s	20	18	ns	<code>clflushopt+sfence</code>
o_r	20	438	ns	<code>clflush+mfence+load</code>
L	150	142	ns	Ping-pong RTT
g	4.0	4.6	ns	Streaming flush
BW	16.0	13.9	GB/s	64B/g

TABLE II: LogP parameters: defaults vs. calibrated values from real CXL hardware (2 hosts, 1 switch).

caps congestion at 100 ns even at full utilization, while real hardware shows over 1,400 ns. The calibrated model matches measurements within 10% across all utilization levels, and `FabricLinkConfig` is updated with the fitted parameters automatically after calibration completes.

Figure 9a shows the CDF of individual CXL operations, confirming that o_s (flush) is tightly distributed around 18 ns while o_r (invalidate) has a wider tail to ~ 700 ns.

Coherence protocol validation. The calibration suite also validates cross-host locking strategies: Lamport bakery (pure stores + flush) tests whether `clflush+sfence` provides sequential consistency across the switch; `flush+CAS` tests device-serialized atomics; and an ownership lock tests pure software coherence. Results inform which coherence protocol OCEAN default to when hardware coherence is unavailable.

VII. CONCLUSION

We presented OCEAN, a high-fidelity open-source CXL 3.0 emulation framework supporting up to 16 hosts with active latency injection, token-bucket bandwidth enforcement, MESI software cache coherence, and full GFAM/MH-SLD/DCD device models built on QEMU 10.0 and Linux 6.16. Fidelity against real CXL Type-3 hardware confirms a $\sim 2.9\times$ latency ratio driven by IPC overhead, not modeling error. Case studies on Tigon database workloads quantify, for the first time, the throughput budget that hardware cache coherence unlocks for cross-host transactional processing — an 11 $\times$ improvement at full HCC coverage. A design-space sweep of twelve memory-placement policies on GROMACS demonstrates OCEAN’s value as an exploration platform, revealing up to 7% performance variation from policy choice alone.

REFERENCES

- [1] Y. Yang, P. Safayenkoo, J. Ma, T. A. Khan, and A. Quinn, “CXLMemSim: A pure software simulated CXL.mem for performance characterization,” in *The Fifth Young Architect Workshop (YArch’23)*, 2023, [Online]. Available: <https://doi.org/10.48550/arXiv.2303.06153>.
- [2] Y. Wang, L. Wu, W. Hong, Y. Ou, Z. Wang, S. Gao, J. Zhang, S. Ma, D. Dong, X. Qi, M. Lai, and N. Xiao, “Cxl-dmsim: A full-system cxl disaggregated memory simulator with comprehensive silicon validation,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 45, no. 4, pp. 1787–1801, 2026. doi: 10.1109/T-CAD.2025.3607145.
- [3] K. Liu, “Cxl-emulator-qemu.” [Online]. Available: <https://github.com/linuslau/CXL-Emulator-QEMU>
- [4] R. Gond and P. Kulkarni, “emucxl: an emulation framework for cxl-based disaggregated memory applications,” *arXiv preprint arXiv:2404.08311*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2404.08311>
- [5] CXL Consortium, “Compute express link (cxl) specification, version 3.0,” CXL Consortium, Tech. Rep., August 2022, accessed: 2026-04-03. [Online]. Available: <https://computeexpresslink.org/download-the-specification/>
- [6] X. Wang, J. Liu, J. Wu, S. Yang, J. Ren, B. Shankar, and D. Li, “Performance characterization of cxl memory and its use cases,” in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2025, pp. 1048–1061. [Online]. Available: <https://doi.org/10.1109/IPDPS64566.2025.00097>
- [7] X. Wang, B. Ma, J. Kim, B. Koh, H. Kim, and D. Li, “cmpi: Using cxl memory sharing for mpi one-sided and two-sided inter-node communications,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 2216–2232. [Online]. Available: <https://doi.org/10.1145/3712285.3759816>
- [8] H. Li, D. S. Berger, S. Novakovic, L. Hsu, D. Ernst, P. Zardoshti, M. Shah, S. Rajadnya, S. Lee, I. Agarwal *et al.*, “Pond: Cxl-based memory pooling systems for cloud platforms,” in *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2023. [Online]. Available: <https://doi.org/10.1145/3575693.3578835>
- [9] Y. Huang, H. Chen, N. Ni, Y. Sun, V. Chidambaram, D. Tang, and E. Witchel, “Tigon: A distributed database for a {CXL} pod,” in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025, pp. 109–128. [Online]. Available: <https://dl.acm.org/doi/10.5555/3767901.3767908>
- [10] S. Vanecek, M. Turner, M. Gajbe, M. Wolf, and M. Schulz, “Modeling the potential of message-free communication via cxl. mem,” in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, 2026, pp. 258–270. [Online]. Available: <https://doi.org/10.1145/3773656.3773661>
- [11] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj *et al.*, “The gem5 simulator: Version 20.0+,” *arXiv preprint arXiv:2007.03152*, 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2007.03152>
- [12] Y. Chen, X. Dong, Z. Zhou, Z. Wang, J. Zhang, and G. Sun, “Exploring memory tiering systems in the cxl era via fpga-based emulation and device-side management,” in *2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2025, pp. 71–83. [Online]. Available: <https://doi.org/10.1109/PACT65351.2025.00018>
- [13] D. D. Sharma, R. Blankenship, and D. S. Berger, “An introduction to the compute express link (CXL) interconnect,” *ACM Computing Surveys*, vol. 56, no. 11, pp. 290:1–290:37, 2024, [Online]. Available: <https://doi.org/10.1145/3669900>.
- [14] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan, “Tpp: Transparent page placement for cxl-enabled tiered-memory,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 742–755. [Online]. Available: <https://doi.org/10.1145/3582016.3582063>
- [15] A. Raybuck, T. Stamler, W. Zhang, M. Erez, and S. Peter, “Hemem: Scalable tiered memory management for big data applications and real nvme,” in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, ser. SOSP ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 392–407. doi: 10.1145/3477132.3483550. [Online]. Available: <https://doi.org/10.1145/3477132.3483550>
- [16] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, “The gem5 simulator,” *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011. [Online]. Available: <https://doi.org/10.1145/2024716.2024718>
- [17] fadzipper, “Gem5 cxl version,” 2022. [Online]. Available: <https://github.com/fadzipper/gem5-cxl/compare/stable...cxl.mem-dev>
- [18] Y. An, S. Yi, B. Mao, Q. Li, M. Zhang, K. Zhou, N. Xiao, G. Sun, X. Wang, Y. Luo *et al.*, “A novel extensible simulation framework for cxl-enabled systems,” *arXiv preprint arXiv:2411.08312*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2411.08312>
- [19] A. Puri, K. Bellamkonda, K. Narreddy, J. Jose, V. Tamarapalli, and V. Narayanan, “Dracksim: simulating cxl-enabled large-scale disaggregated memory systems,” in *Proceedings of the 38th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2024, pp. 3–14. [Online]. Available: <https://doi.org/10.1145/3615979.3656059>
- [20] S. Kim, J. Kang, K. Kim, S. Lee, and B. Nam, “Cxlsim: A simulator for cxl memory expander,” in *2025 IEEE International Conference on Big Data and Smart Computing (BigComp)*. IEEE, 2025, pp. 156–159. doi: 10.1109/BigComp64353.2025.00042.
- [21] J. Wahlgren, M. Gokhale, and I. B. Peng, “Evaluating emerging cxl-enabled memory pooling for hpc systems,” in *2022 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*. IEEE, 2022, pp. 11–20. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.02682>
- [22] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong *et al.*, “Demystifying cxl memory with genuine cxl-ready systems and devices,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 105–121. [Online]. Available: <https://doi.org/10.1145/3613424.3614256>
- [23] J. Liu, H. Hadian, Y. Wang, D. S. Berger, M. Nguyen, X. Jian, S. H. Noh, and H. Li, “Systematic cxl memory characterization and performance analysis at scale,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 1203–1217. [Online]. Available: <https://doi.org/10.1145/3676641.3715987>
- [24] S.-P. Yang, M. Kim, S. Nam, J. Park, J. yong Zhang, E. H. Nam, E. Lee, S. Lee, and B. S. Kim, “Overcoming the memory wall with CXL-Enabled SSDs,” in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 601–617. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/yang-shao-peng>
- [25] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, “Pim is all you need: A cxl-enabled gpu-free system for large language model inference,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 862–881. [Online]. Available: <https://doi.org/10.1145/3676641.3716267>
- [26] Y. Shan, Y. Huang, Y. Chen, and Y. Zhang, “[LegoOS]: A disseminated, distributed {OS} for hardware resource disaggregation,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 69–87. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/shan>
- [27] T. Ma, Z. Liu, C. Wei, J. Huang, Y. Zhuo, H. Li, N. Zhang, Y. Guan, D. Niu, M. Zhang *et al.*, “{HydraRPC}::{RPC} in the {CXL} era,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 387–395. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/ma>
- [28] D. Gouk, S. Lee, M. Kwon, and M. Jung, “Direct access,{High-Performance} memory disaggregation with {DirectCXL},” in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 287–294.
- [29] J. Wahlgren, G. Schieffer, M. Gokhale, and I. B. Peng, “A quantitative approach for adopting disaggregated memory in HPC systems,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023, pp. 1–14. [Online]. Available: <https://doi.org/10.1145/3581784.3607108>.
- [30] A. Devalapally, M. Halappanavar, B. Jacob, V. Narayanan, and A. Marquez, “Revisiting pebble games for modeling and efficient use of disaggregated memory systems,” in *Proceedings of the International*

- Symposium on Memory Systems*, 2025, pp. 8–14. [Online]. Available: <https://doi.org/10.1145/3767110.3767118>
- [31] ScaleMP, “vSMP Foundation: Versatile SMP virtualization for high-end computing,” ScaleMP Inc., original site at <https://www.scalemp.com/products/vsmp-foundation/> (domain no longer active); archived via the Wayback Machine. [Online]. Available: https://web.archive.org/web/20230000000000*/https://www.scalemp.com/products/vsmp-foundation/
 - [32] M. Larabel, “Benchmarking Numascale’s NumaConnect: 256 AMD cores connected,” Phoronix, Mar. 2014, accessed: 2026-03-17. [Online]. Available: <https://www.phoronix.com/news/MTY0NzA>
 - [33] A. Koshiba, T. Hirofuchi, R. Takano, and M. Namiki, “A software-based nvm emulator supporting read/write asymmetric latencies,” *IEICE TRANSACTIONS on Information and Systems*, vol. 102, no. 12, pp. 2377–2388, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1908.02135>
 - [34] H. Volos, G. Magalhaes, L. Cherkasova, and J. Li, “Quartz: A lightweight performance emulator for persistent memory software,” in *Proceedings of the 16th Annual Middleware Conference*, 2015, pp. 37–49. [Online]. Available: <https://doi.org/10.1145/2814576.2814806>
 - [35] G. Zhu, K. Lu, X. Wang, X. Zhou, and Z. Shi, “Building emulation framework for non-volatile memory,” *IEEE Access*, vol. 5, pp. 21 574–21 584, 2017. doi: 10.1109/ICDCSW.2017.12.
 - [36] B. Gregg, *BPF Performance Tools*. Addison-Wesley Professional, 2019.
 - [37] V. M. Weaver, “Advanced hardware profiling and sampling (pebs, ibs, etc.): creating a new papi sampling interface,” Technical Report UMAINE-VMWTR-PEBS-IBS-SAMPLING-2016-08. University of Maine, Tech. Rep., 2016.
 - [38] I. Team, “Timed process event based sampling (tpebs),” 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/timed-process-event-based-sampling-tpebs.html>
 - [39] D. Bakhvalov, “Advanced profiling topics. pebs and lbr,” 2018. [Online]. Available: <https://easyperf.net/blog/2018/06/08/Advanced-profiling-topics-PEBS-and-LBR#processor-event-based-sampling-pebs>
 - [40] D. Culler, R. Karp, D. Patterson, A. Sahay, K. E. Schauer, E. Santos, R. Subramonian, and T. von Eicken, “LogP: Towards a realistic model of parallel computation,” in *Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOP)*. ACM, 1993, pp. 1–12. [Online]. Available: <https://doi.org/10.1145/155332.155333>
 - [41] J. McCalpin, “STREAM,” Accessed: Feb. 10, 2025. [Online]. Available: <https://www.cs.virginia.edu/stream/ref.html>
 - [42] L. McVoy and C. Staelin, “LMbench,” Accessed: Feb. 10, 2025. [Online]. Available: <https://lmbench.sourceforge.net>
 - [43] Network-Based Computing Laboratory, *OSU Micro-Benchmarks*, The Ohio State University, 2024, version 7.4. [Online]. Available: <https://mvapich.cse.ohio-state.edu/benchmarks/>
 - [44] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *Proc. 1st ACM Symp. Cloud Comput. (SOCC)*, 2010, pp. 143–154. doi: 10.1145/1807128.1807152.
 - [45] GROMACS Development Team, *GROMACS 2026.1 Release*, 2026. doi: 10.5281/zenodo.18887033., software and Documentation. [Online]. Available: <https://doi.org/10.5281/zenodo.18887033>
 - [46] W. Huang, M. Sha, M. Lu, Y. Chen, B. He, and K.-L. Tan, “Bandwidth expansion via cxl: A pathway to accelerating in-memory analytical processing,” *Proceedings of the VLDB Endowment. ISSN*, vol. 2150, p. 8097, 2024. doi: 10.14778/3683467.3683470.
 - [47] M. Arif, K. Assogba, M. M. Rafique, and S. Vazhkudai, “Exploiting CXL-based memory for distributed deep learning,” in *Proceedings of the 51st International Conference on Parallel Processing (ICPP)*, 2022, [Online]. Available: <https://doi.org/10.1145/3545008.3545054>
 - [48] Q. Wang, Y. Lu, E. Xu, J. Li, Y. Chen, and J. Shu, “Concordia: Distributed shared memory with In-Network cache coherence,” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 277–292, [Online]. Available: <https://www.usenix.org/conference/fast21/presentation/wang>.